BÁO CÁO THỰC HÀNH
CẤU TRÚC CÂY BST VÀ AVL

Cấu trúc dữ liệu và giải thuật • Phần 3 • Ngày 02 tháng 10 năm 2026

Bộ bài làm triển khai cây nhị phân tìm kiếm, cây AVL, các bài mở rộng từ cơ bản đến nâng cao và phần suy luận của giáo trình. Kết quả gồm 13 chương trình C++ độc lập cùng báo cáo giải thích thuật toán, cây minh họa, độ phức tạp và kết quả kiểm thử.

Các chương trình dùng nút và con trỏ, không dùng map hoặc set để thay thế cây. Kết quả thực nghiệm cho thấy AVL giữ chiều cao thấp khi chèn khóa tăng dần; BST thường có thể suy biến thành một nhánh. Toàn bộ chương trình đã được biên dịch và chạy trên MinGW GCC 6.3.0 với chuẩn C++11.

Danh mục mã nguồn

File trong thư mục source	Nội dung
Bai09_BST.cpp	BST cơ bản và menu tương tác
Bai09_MoRong.cpp	Bài mở rộng 1–6 và 9; bổ sung k nhỏ nhất và sao chép
Bai09_KhoaTrung.cpp	Bài mở rộng 8 với trường count
Bai10_AVL.cpp	AVL, bài mở rộng 7, xóa và kiểm tra AVL
Bai11_SoSanh.cpp	Đo chiều cao và số nút thăm khi tìm kiếm
N1_ThuatNgu_Duyet.cpp	Thuật ngữ, duyệt đệ quy và stack
N2_Chen_KiemBST.cpp	Chèn từng bước và phản ví dụ kiểm tra BST
N3_XoaBST.cpp	Xóa lá, một con và gốc hai con
N4_CayBieuThuc.cpp	Dựng và tính cây biểu thức từ hậu tố
N5_BonXoay.cpp	Bốn xoay và vết chèn AVL từ 1 đến 7
N6_Mang_TinhChat.cpp	Biểu diễn mảng và đếm loại nút
N7_DungCay.cpp	Dựng từ NLR và LNR, khôi phục BST
N8_TuDien.cpp	Từ điển BST và AVL với khóa chuỗi

1 Quy ước và hiệu chỉnh mã mẫu

Quy ước xuyên suốt

Gốc có mức 1. Cây rỗng có chiều cao 0, nút lá có chiều cao 1; chiều cao đếm số nút trên đường dài nhất từ gốc đến lá. BF = h(phải) − h(trái): lệch trái mang dấu âm, lệch phải mang dấu dương. BST chuẩn không nhận khóa trùng; bài tần suất lưu số lần xuất hiện trong count của một nút duy nhất.

Một "so sánh" trong thực nghiệm là một lần đối chiếu ba ngả khóa cần tìm với khóa tại nút, tương đương số nút được thăm. Không đếm lần chạm con trỏ null. Đây không phải tổng số toán tử == và < mà mã máy thực hiện. Quy ước giống nhau cho BST và AVL.

Các điểm cần sửa trong giáo trình

Vị trí PDF	Điểm chưa nhất quán và cách xử lý
Trang 6 và 10	Trong cây có nút 4, chèn 5 phải đi 8 → 3 → 6 → 4 rồi gắn 5 bên phải 4. Tìm 5 cũng phải đi qua 4 trước khi gặp null.
Trang 14	Khoảng mở dùng INT_MIN và INT_MAX sẽ loại nhầm khóa bằng hai biên. IsBST dùng giới hạn long long để nhận toàn bộ miền int.
Trang 15, 25, 26, 40	Tài liệu trộn cách đếm cạnh và đếm nút. Bài làm theo quy ước môn tại trang 40: rỗng 0, lá 1. BST 10 khóa tăng có h = 10.
Trang 22	Hình được gọi là AVL hợp lệ thực ra mất cân tại nút 10: nhánh phải 14 → 13 có h = 2, trái rỗng, BF = 2.
Trang 30 và 31	Điều kiện xóa AVL dùng dấu của quy ước BF ngược. Bài làm dùng BF < −1 cho lệch trái và BF > 1 cho lệch phải.
Trang 36	Tìm thấy ngay tại gốc có thời gian tốt nhất O(1). O(log n) là bảo đảm xấu nhất của tìm kiếm AVL, không phải tốt nhất.
Trang 45 và 47	NLR + LRN không luôn xác định duy nhất cây có nút một con. NLR + LNR xác định duy nhất nếu các khóa phân biệt.

Quản lý bộ nhớ và phạm vi chạy

Các hàm thay đổi cấu trúc trả về gốc mới; nơi gọi luôn gán lại root. Clear và clearAVL giải phóng các nút khi kết thúc hoặc nạp lại mẫu. Các phép duyệt và BST cơ bản dùng đệ quy để bám sát giáo trình; cây suy biến rất lớn có thể làm cạn stack. Thực nghiệm kích thước tối đa 1.000 khóa, không suy rộng thành cam kết chạy được cây suy biến hàng trăm nghìn nút.

2 Bài 9 Cây nhị phân tìm kiếm

Mã nguồn: Bai09_BST.cpp. Cấu trúc TNode gồm data, pLeft và pRight. Cây rỗng biểu diễn bằng nullptr. Khi chèn hoặc tìm, mỗi phép so sánh chỉ chọn một nhánh vì toàn bộ khóa trái nhỏ hơn gốc và toàn bộ khóa phải lớn hơn gốc.

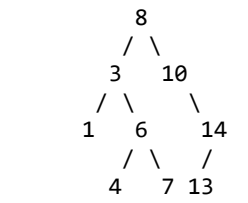
Thuật toán chính

Insert tạo nút ở vị trí rỗng, trả nguyên cây nếu gặp khóa trùng. Search có bản đệ quy và SearchIterative dùng vòng lặp. Delete xử lý nút lá và nút một con bằng cách trả về con còn lại; với hai con, sao chép khóa nhỏ nhất của cây phải rồi xóa nút thế mạng ở cây phải.

```
if (!r->pLeft || !r->pRight) {
    TNode* child = r->pLeft ? r->pLeft : r->pRight;
    delete r;
    return child;
}
TNode* next = FindMin(r->pRight);
r->data = next->data;
r->pRight = Delete(r->pRight, next->data);
```

Kết quả mẫu

Dãy chèn: 8, 3, 10, 1, 6, 14, 4, 7, 13. Cây có 9 nút và chiều cao 4. IsBST trả true.




```
NLR   8 3 1 6 4 7 10 14 13
LNR   1 3 4 6 7 8 10 13 14
LRN   1 4 7 6 3 13 14 10 8
Level 8 3 10 1 6 14 4 7 13
```

Kiểm tra tính chất và chi phí

IsBST truyền khoảng hợp lệ từ tổ tiên xuống mọi nút, không chỉ so sánh con trực tiếp. Chèn, tìm, xóa tốn O(h), xấu nhất O(n). Các phép duyệt, đếm nút, tính chiều cao và kiểm tra BST tốn O(n); đệ quy dùng O(h) stack. Level-order dùng queue với bộ nhớ O(w), w là bề rộng lớn nhất. PrintTree thăm O(n) nút nhưng độ dài chuỗi thụt lề có thể làm tổng số ký tự in lên đến O(nh).

Menu có chèn, xóa, tìm, bốn phép duyệt, vẽ cây, thống kê, kiểm tra BST, nạp cây mẫu và xóa hết. Nhập chữ ở vị trí số được báo lỗi và cho nhập lại; EOF kết thúc an toàn.

3 Bài mở rộng mức cơ bản và trung bình

Mã nguồn: Bai09_MoRong.cpp. Chương trình chạy lần lượt các bài và in kết quả; đối số dòng lệnh tùy chọn là tên file lưu cây. Nếu không truyền, chương trình tạo cay_bst.txt trong thư mục chạy.

Bài	Cách làm và độ phức tạp
1 Gần nhất	Đi theo đường Search, giữ khóa có khoảng cách nhỏ nhất. Nếu hòa, chọn khóa nhỏ hơn. Ép sang long long trước khi trừ để tránh tràn int. Thời gian O(h), bộ nhớ O(1).
2 Min và max	Đi hết nhánh trái hoặc phải; cây rỗng trả nullptr. O(h) thời gian, O(1) bộ nhớ.
3 Complete	BFS gồm cả vị trí null. Sau khoảng trống đầu tiên, nếu còn nút thật thì không complete. Tầng cuối phải lấp từ trái sang phải. O(n) thời gian và bộ nhớ.
4 Cân bằng BST	LNR lấy mảng tăng; giải phóng cây cũ; lấy phần tử giữa mỗi đoạn làm gốc cây mới. O(n) thời gian, O(n) bộ nhớ. Cây mới cân tại thời điểm dựng, không tự cân ở lần chèn sau.
5 Lưu và khôi phục	Lưu NLR. ReadPre đọc theo khoảng hợp lệ, mỗi khóa dùng một lần, O(n); từ chối khóa trùng, thứ tự không hợp lệ và token không phải int. Cây BST không trùng khôi phục đúng hình dạng.
6 Phạm vi	Duyệt trái nếu lo < data; in nếu lo ≤ data ≤ hi; duyệt phải nếu data < hi. O(h + k) với k khóa trả về, xấu nhất O(n). lo > hi trả dãy rỗng.
9 Vẽ console	DrawTree gọi PrintTree. Ký hiệu ROOT, L---, R--- và null phân biệt rõ cạnh trái, phải. Không cần thư viện giao diện.

Kết quả đã chạy

```
Gan 5 nhat: 4
Min=1 Max=14
Complete=0
Range [4,10]: 4 6 7 8 10
K=4: 6
Copy dung=1
Luu/doc giu hinh=1
Height truoc=7
Height sau=3 Complete=1
```

Hai chức năng bổ sung phục vụ C13 và C14: KthSmallest dùng stack duyệt LNR và dừng tại nút thứ k; k ≤ 0 hoặc k vượt số nút trả nullptr. CopyTree tạo gốc rồi sao chép hai cây con; SameTree so sánh cả khóa lẫn cấu trúc, nên kiểm tra lưu/đọc không chỉ dựa trên dãy LNR.

4 Khóa trùng và cấu trúc AVL

Bài mở rộng 8 Lưu tần suất

Mã nguồn: Bai09_KhoaTrung.cpp. Mỗi khóa chỉ có một nút, count ≥ 1. Chèn trùng tăng count. Xóa một lần giảm count nếu count > 1; chỉ xóa vật lý khi count về 0. Thời gian mỗi thao tác O(h), số nút bằng số khóa phân biệt.

Khi xóa nút hai con, phải chuyển cả data và count của nút thế mạng. Sau đó xóa toàn bộ nút thế mạng cũ bằng eraseAll=true. Nếu chỉ giảm count của nút thế mạng, hai nút cùng khóa sẽ cùng tồn tại và phá bất biến BST.

```
Chen: 8 3 10 3 8 8 9 9
LNR: 3(count=2) 8(count=3) 9(count=2) 10(count=1)
Xoa 8 ba lan: 3(count=2) 9(count=2) 10(count=1)
Xoa 3 mot lan: 3(count=1) 9(count=2) 10(count=1)
Xoa 42: cay khong doi
```

Bài 10 và bài mở rộng 7 Cây AVL

Mã nguồn: Bai10_AVL.cpp. AVLNode bổ sung height để lấy chiều cao cây con trong O(1). Sau khi chèn hoặc xóa như BST, cập nhật height và gọi rebalance trên đường quay lui. Gốc của cây con có thể thay đổi sau xoay nên luôn nhận giá trị trả về.

Tình trạng	Điều kiện theo BF phải trừ trái	Xử lý
LL	BF(gốc) < −1, BF(trái) ≤ 0	Xoay phải gốc
LR	BF(gốc) < −1, BF(trái) > 0	Xoay trái con trái rồi phải gốc
RR	BF(gốc) > 1, BF(phải) ≥ 0	Xoay trái gốc
RL	BF(gốc) > 1, BF(phải) < 0	Xoay phải con phải rồi trái gốc

Bảng áp dụng cho cả chèn và xóa. Dấu bằng trong BF(con) = 0 rất quan trọng khi xóa; không thể dựa vào khóa vừa xóa để suy ra hướng mất cân bằng. Một lần xóa có thể cần cân bằng nhiều tổ tiên.

Bất biến được kiểm tra

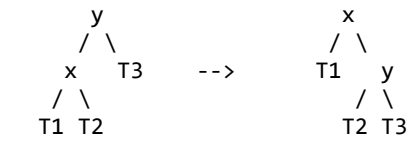
IsAVL đồng thời kiểm tra thứ tự BST, độ chênh chiều cao không quá 1 và height lưu tại từng nút bằng chiều cao thật. Hàm tính lại chiều cao từ lá lên, vì chỉ kiểm BF dựa trên height đã sai có thể bỏ lọt lỗi. Kiểm tra toàn cây tốn O(n); chèn, xóa, tìm kiếm AVL có chi phí xấu nhất O(log n).

Menu cung cấp chèn, xóa, tìm, bốn phép duyệt, hình cây kèm h và BF, kiểm tra AVL, bật/tắt vết xoay và xóa cây. Chế độ --demo minh họa dãy 10, 20, 30, 40, 50, 25, thêm 15 và xóa 20.

5 Phép xoay và xóa AVL

Giữ cây con trung gian

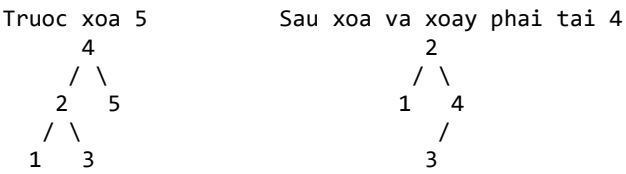
Xoay phải tại y: x là con trái của y, T2 là con phải của x. Cho x làm gốc mới, y làm con phải x và gắn T2 thành con trái y. Thứ tự khóa vẫn là T1 < x < T2 < y < T3. Xoay trái là phép đối xứng.




```
// Xoay phai: cap nhat nut di xuong truoc
y->height = 1 + max(getHeight(y->left), getHeight(y->right));
x->height = 1 + max(getHeight(x->left), getHeight(x->right));
// Xoay trai: thu tu doi xung
x->height = 1 + max(getHeight(x->left), getHeight(x->right));
y->height = 1 + max(getHeight(y->left), getHeight(y->right));
```

Bốn dòng trên là hai dòng cho xoay phải và hai dòng cho xoay trái. Trong mã nguồn, updateHeight đóng gói cùng công thức; thứ tự gọi không thay đổi. Không được cập nhật gốc mới trước nút vừa đi xuống vì chiều cao con của gốc mới khi đó còn cũ.

Ca xóa có BF của con bằng 0



Sau khi xóa 5, nút 4 có BF = −2, nút 2 có BF = 0. Chọn xoay phải đơn. Cây mới có chiều cao 3 và vẫn hợp lệ. Ca đối xứng được kiểm tra với dãy 2, 1, 4, 3, 5 rồi xóa 1.

Kết quả kiểm chứng

Bốn dãy LL, RR, LR, RL đều cho gốc 20, trái 10, phải 30, chiều cao 2. Kiểm thử đã đối chiếu 10.000 thao tác chèn/xóa với một vector khóa chuẩn; sau mỗi thao tác kiểm thứ tự, chiều cao thực và cân bằng toàn cây. Một phép tăng sai height có chủ ý cũng bị IsAVL phát hiện.

6 Bài 11 So sánh BST và AVL

Mã nguồn: Bai11_SoSanh.cpp. Hai cây nhận cùng dãy khóa và tìm cùng khóa. Báo cáo chiều cao và số nút thăm; không dùng một lần đo clock để kết luận. Các hoán vị dùng mt19937 với seed 7, 42, 2026. Trong chương trình, mỗi lần shuffle tác động lên vector hiện tại; chạy cùng chương trình và thư viện cho cùng chuỗi thí nghiệm.

Dữ liệu	n	Khóa	h BST	h AVL	SS BST	SS AVL
Mau	10	17	10	4	9	3
Tang	100	100	100	7	100	7
Khong thay	100	101	100	7	100	7
Giam	100	1	100	7	100	7
Tron seed=7	100	100	13	8	5	7
Tron seed=42	100	100	15	8	4	5
Tron seed=2026	100	100	13	8	5	7
Tang	1000	1000	1000	10	1000	10
Khong thay	1000	1001	1000	10	1000	10
Giam	1000	1	1000	10	1000	10
Tron seed=7	1000	1000	22	12	5	10
Tron seed=42	1000	1000	20	12	8	11
Tron seed=2026	1000	1000	23	12	5	9

Với dãy 1, 3, 5, 7, 9, 11, 13, 15, 17, 19, BST là nhánh phải cao 10; tìm 17 thăm 9 nút. AVL cao 4, tìm 17 theo đường 7 → 15 → 17, chỉ thăm 3 nút. Tỉ lệ số nút thăm là 3 lần trong ca này.

Với 1.000 khóa tăng, BST cao 1.000 và tìm khóa cuối thăm 1.000 nút; AVL cao 10 và thăm 10 nút. Tổng chi phí dựng BST từ dãy tăng là $O(n^2)$, trong khi dựng AVL là $O(n \log n)$.

Ở các hoán vị ngẫu nhiên, khóa lớn nhất đôi khi nằm nông hơn trong BST nên số nút thăm của riêng truy vấn đó nhỏ hơn AVL. Không thể suy từ một truy vấn rằng BST tốt hơn tổng thể. Điểm mạnh của AVL là giới hạn chiều cao và chi phí xấu nhất; xoay và cập nhật height có chi phí phụ. Muốn đo thời gian trung bình cần nhiều truy vấn và nhiều lần lặp.

7 N1 và N2 Thuật ngữ và chèn cây

N1 Bốn phép duyệt

Mã nguồn: N1_ThuatNgu_Duyet.cpp. Dùng cây BST tạo bởi 8, 3, 10, 1, 6, 14, 4. Gốc là 8; lá là 1, 4, 14; cha của 4 là 6; mức của 4 bằng 4; bậc của 3 bằng 2. Có $n_0 = 3$ và $n_2 = 2$ nên $n_0 = n_2 + 1$.

```
NLR   8 3 1 6 4 10 14
LNR   1 3 4 6 8 10 14
LRN   1 4 6 3 14 10 8
Level 8 3 10 1 6 14 4

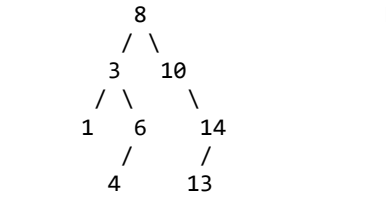
Cay 1 -> 2 -> 3 -> 4 chi co con phai
NLR = LNR = Level = 1 2 3 4
LRN = 4 3 2 1; chieu cao = 4
```

NLR không đệ quy dùng stack: lấy gốc ra, thăm, đẩy phải rồi đẩy trái để trái được lấy trước. Cây suy biến giống danh sách liên kết ở chỗ mỗi nút chỉ dẫn đến một nút kế tiếp trên nhánh; nút cây vẫn có hai trường con trỏ và phải giữ ý nghĩa trái/phải.

Cây thư mục có thể có số con tùy ý. Cây nhị phân có tối đa hai con có thứ tự trái và phải. Hai con trỏ của TNode biểu diễn trực tiếp hai quan hệ này. Cây tổng quát có thể mã hóa bằng con trái là con đầu tiên và con phải là anh em kế tiếp; khi đó ý nghĩa con trỏ đã khác BST.

N2 Chèn tiếp 14 và 13

Mã nguồn: N2_Chen_KiemBST.cpp. Sau dãy 8, 3, 10, 1, 6, 4, chèn 14 theo 8 → 10 → phải; chèn 13 theo 8 → 10 → 14 → trái. Chèn lại 8 bị bỏ qua, tổng số nút vẫn bằng 8.




```
Phan vi du
    8
   / \
  3  10
 / \
1  6
```

```
NLR   8 3 1 6 4 10 14 13
LNR   1 3 4 6 8 10 13 14
LRN   1 4 6 3 13 14 10 8
Level 8 3 10 1 6 14 4 13
```

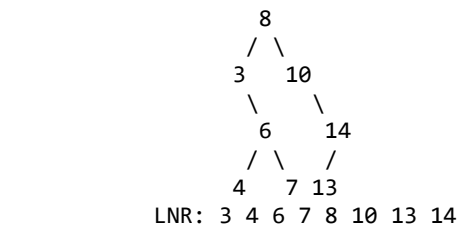
Phản ví dụ trả IsBST=false vì 9 nằm trong cây trái của 8 nhưng lớn hơn 8. Với cổng tra 80.000 MSSV thêm theo mã tăng, BST thường không có bảo đảm phù hợp: h có thể bằng n. Nếu cần thêm trực tuyến và tra cứu ổn định, chọn AVL; khi danh sách gần như bất biến, mảng đã sắp xếp và tìm nhị phân cũng là phương án đáng xét.

8 N3 Xóa BST đủ ba trường hợp

Mã nguồn: N3_XoaBST.cpp. Thực hiện tuần tự trên cây 8, 3, 10, 1, 6, 14, 4, 7, 13; không dựng lại cây giữa ba lần xóa.

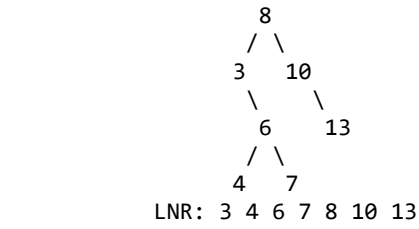
Xóa 1 là nút lá

Cha là 3. Hàm trả nullptr cho nhánh trái của 3, sau đó giải phóng nút 1. Các liên kết khác giữ nguyên.



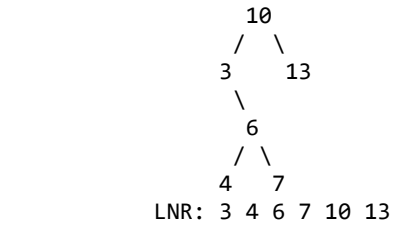
Xóa 14 có một con

Nút 14 có con trái 13. Gán liên kết phải của 10 sang 13 rồi giải phóng 14. Nút 13 được giữ lại, không bị xóa theo cha.



Xóa 8 là gốc có hai con

Nút thế mạng là 10, nhỏ nhất của cây con phải. Sao chép 10 vào gốc; xóa nút 10 cũ bằng cách đưa con phải 13 lên. Toàn bộ cây con trái chứa 3, 4, 6, 7 vẫn được nối với gốc mới.



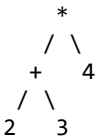
Cả ba bước đều qua IsBST. Xóa mất O(h), kể cả khi tìm nút thế mạng; trường hợp xấu của BST không cân là O(n). Trong quản lý sinh viên, xóa khóa của người thôi học phải giữ các hồ sơ còn lại và ưu tiên cấu trúc phù hợp với thao tác tra cứu thường xuyên.

9 N4 Cây biểu thức và stack

Mã nguồn: N4_CayBieuThuc.cpp. Stack tự cài bằng danh sách liên kết, lưu con trỏ Expr. Token cách nhau bằng khoảng trắng; hỗ trợ số, các toán tử hai ngôi +, -, *, /. Toán tử âm một ngôi không tách riêng; số âm như -3 được nhận là một token số.

Dựng cây từ hậu tố

Với 2 3 + 4 *, đẩy lá 2 và 3. Gặp +, pop 3 làm con phải và 2 làm con trái, đẩy cây +. Đẩy lá 4. Gặp *, pop 4 làm con phải và cây + làm con trái. Cuối cùng stack phải còn đúng một cây.




```
LNR co ngoac: ((2+3)*4)
LRN hau to:  2 3 + 4 *
NLR tien to: * + 2 3 4
Gia tri: 20
```

Duyệt hậu tự phù hợp để tính vì hai toán hạng đã được tính trước khi áp dụng toán tử ở gốc. Duyệt LNR phải thêm ngoặc để giữ đúng cấu trúc; nếu chỉ in 2+3*4 sẽ thay đổi nghĩa biểu thức.

Kiểm tra lỗi và thứ tự toán hạng

Đầu vào	Kết quả
2 3 + 4 *	20
2 3 4 * +	14
8 2 -	6, kiểm tra không đảo trái và phải
8 2 /	4, kiểm tra không đảo trái và phải
2 +	Báo thiếu toán hạng
2 3	Báo thừa toán hạng
2 0 /	Báo chia cho 0
Chuỗi rỗng hoặc abc	Báo biểu thức rỗng hoặc token số không hợp lệ

Dựng cây tốn O(t) với t token, duyệt và tính tốn O(t); bộ nhớ O(t), stack đệ quy đánh giá O(h). Khi đầu vào sai, stack giải phóng các cây đang sở hữu. Khi tính lỗi, main giải phóng cây trước khi trả mã thoát 1. Kết quả vượt miền hữu hạn của double cũng được từ chối.

```
g++ -std=c++11 N4_CayBieuThuc.cpp -o N4.exe
.\N4.exe "2 3 + 4 *"
.\N4.exe "8 2 -"
```

10 N5 Bốn trường hợp xoay AVL

Mã nguồn: N5_BonXoay.cpp. Các hình dưới thể hiện cây ngay trước khi khôi phục cân bằng; các cây con không vẽ đều rỗng. Cả bốn dãy cho cùng cây cuối gốc 20, hai con 10 và 30.

LL và RR

```
LL: 30,20,10                    RR: 10,20,30
       30                              10
      / \                             / \
     20 T3                           T1 20
    / \                                / \
   10 T2                              T2 30
   T1 la cay goc 10                    T3 la cay goc 30

Xoay phai tai 30                 Xoay trai tai 10
        20                               20
       /  \                             /  \
      10  30                           10  30
```

LL: T2 = con phải 20 chuyển sang con trái 30; T3 vẫn là con phải 30. RR: T2 = con trái 20 chuyển sang con phải 10; T1 vẫn là con trái 10. Trong hai dãy ba khóa, T2 rỗng.

LR và RL

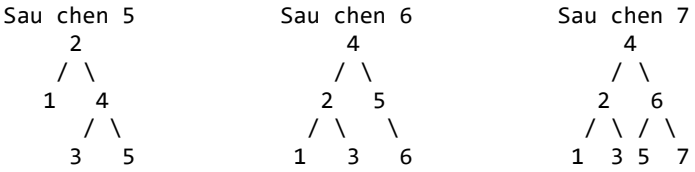
```
LR: 30,10,20                 RL: 10,30,20
       30                           10
      /  \                         /  \
     10  T4                       T1  30
    /  \                             /  \
   T1  20                           20  T4
      /  \                         /  \
     T2  T3                       T2  T3

Xoay trai 10, phai 30         Xoay phai 30, trai 10
             20                           20
            /  \                         /  \
           10  30                       10  30
          / \  / \                     / \  / \
         T1 T2 T3 T4                  T1 T2 T3 T4
```

Xoay kép có bốn cây con biên T1–T4. Sau cân bằng, T2 nằm bên phải 10 và T3 nằm bên trái 30; không được bỏ cây con trung gian. Ở dãy ba khóa, cả bốn cây con đều rỗng. Chương trình còn in cây trước xoay, tên trường hợp và gốc T2 của từng phép xoay đơn để quan sát lúc T2 khác rỗng.

11 N5 Vết chèn từ 1 đến 7

Chèn	Mất cân và xử lý	Gốc và chiều cao sau bước
1	Tạo lá, không xoay	Gốc 1, h = 1
2	BF(1) = 1, không xoay	Gốc 1, h = 2
3	BF(1) = 2, RR; xoay trái tại 1	Gốc 2, h = 2
4	Cân bằng còn hợp lệ, không xoay	Gốc 2, h = 3
5	BF(3) = 2, RR; xoay trái tại 3	Gốc 2, h = 3
6	BF(2) = 2, RR; xoay trái tại 2, T2 là nút 3	Gốc 4, h = 3
7	BF(5) = 2, RR; xoay trái tại 5	Gốc 4, h = 3



Ở bước chèn 6, trước xoay trái: x = 2, y = 4, T2 = 3. Liên kết 2 → phải được đổi từ 4 sang 3, sau đó 4 → trái trỏ về 2. Cập nhật h(2) trước h(4). Đây là trường hợp cần giữ T2 thật, khác các ví dụ ba khóa có T2 rỗng.

AVL cuối có chiều cao 3 và mọi BF bằng 0. BST nhận cùng dãy có chiều cao 7, chỉ có nhánh phải. LNR của cả hai đều là 1 2 3 4 5 6 7, cho thấy thứ tự duyệt đúng chưa đủ chứng minh cây cân bằng.

Phát hiện lỗi quên cập nhật height

Ví dụ 30, 20, 10 rồi 5: nếu xoay phải sau khóa 10 mà không cập nhật height, nút 30 vẫn có height cũ là 3 dù đã thành lá. Khi thêm 5, gốc 20 được tính height = 4 và BF = +1, trong khi thực tế height = 3 và BF = −1. Bất biến chiều cao đã sai dù hình cây vẫn có vẻ đúng.

Không thể khẳng định mọi dãy bốn khóa sẽ làm LL bị chọn thành RR ngay lập tức; lỗi có thể chỉ biểu hiện ở số liệu chiều cao trước khi gây xoay sai ở bước sau. IsAVL tính lại chiều cao nên bắt được lỗi sớm. Hai dòng cập nhật trong mỗi phép xoay tại mục 5 sửa nguyên nhân này.

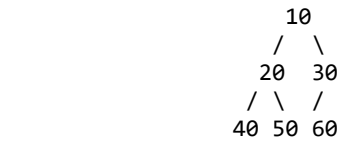
Liên hệ từ điển lớn

Từ điển có 100.000 từ thêm theo thứ tự từ điển khiến BST thường suy biến. Dữ liệu thêm theo thứ tự ngẫu nhiên thường tạo cây thấp hơn nhưng không có bảo đảm xấu nhất như AVL. Nếu việc tìm kiếm xảy ra thường xuyên và dữ liệu tiếp tục tăng, chiều cao được khống chế của AVL đáng giá hơn chi phí cập nhật và xoay khi thêm.

12 N6 và N7 Biểu diễn và dựng cây

N6 Mảng của cây complete

Mã nguồn: N6_Mang_TinhChat.cpp. Mảng 10, 20, 30, 40, 50, 60 biểu diễn các nút theo từng tầng từ trái sang phải. Đây là cây nhị phân complete, không phải yêu cầu BST theo thứ tự khóa.



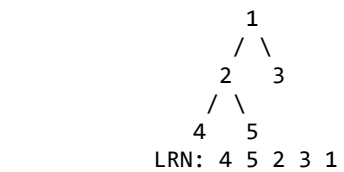

```
Chi so:  0  1  2  3  4  5
Gia tri:10 20 30 40 50 60
Con trai i: 2*i+1; con phai i: 2*i+2
Cha i>0: (i-1)/2; goc khong co cha
```

Nút 20 ở chỉ số 1, con trái ở 3 là 40 và con phải ở 4 là 50. Trên BST mẫu 9 nút: $n_0 = 4$ (1, 4, 7, 13), $n_1 = 2$ (10, 14), $n_2 = 3$ (8, 3, 6). Có $4 = 3 + 1$. Công thức áp dụng cho cây nhị phân không rỗng: số cạnh vừa bằng $n - 1$ vừa bằng $n_1 + 2n_2$.

Heap có dạng complete nên mảng liên tục tránh lỗ trống, cho truy cập cha và con O(1) bằng chỉ số. BST tùy ý có thể rất thưa khi đánh chỉ số theo tầng. Heap vẫn có thể dùng con trỏ và thêm con trỏ cha để truy cập cha O(1), nhưng mất cách định vị ngầm bằng chỉ số và thường tốn bộ nhớ liên kết hơn.

N7 Dựng từ NLR và LNR

Mã nguồn: N7_DungCay.cpp. NLR = 1 2 4 5 3; LNR = 4 2 5 1 3. Gốc là 1; vị trí 1 trong LNR chia trái thành 4 2 5 và phải thành 3. Gốc trái là 2, có trái 4 và phải 5.




```
LRN: 4 5 2 3 1
```

Cài đặt dùng chỉ số đoạn mảng, kiểm tra hai dãy cùng tập khóa phân biệt, rồi tìm vị trí gốc trong đoạn LNR. Thời gian xấu nhất $O(n^2)$ do tìm tuyến tính ở mỗi mức, stack O(h); không dùng map. Cặp dãy mâu thuẫn, khác độ dài hoặc trùng khóa bị từ chối.

Chỉ LNR không xác định duy nhất cây. NLR + LRN cũng có thể mơ hồ khi một nút chỉ có một con; thêm giả thiết cây full và khóa phân biệt mới loại mơ hồ này. Với BST không trùng, chèn lại lần lượt NLR giữ đúng hình cây, vì các đoạn khóa trái/phải đã xác định bởi gốc. Cây nhị phân tổng quát lưu chỉ NLR cần thêm dấu null hoặc thông tin cấu trúc.

13 N8 Chọn cấu trúc theo nghiệp vụ

Hoàn tác nước đi cờ caro

Thao tác chính là lấy lại nước đi vừa thực hiện. Quy tắc đó là LIFO nên dùng Stack. Mỗi lần đánh cờ, push tọa độ và trạng thái cũ cần khôi phục. Khi hoàn tác, pop phần tử trên đỉnh rồi trả bàn cờ về trạng thái trước. Push và pop có chi phí O(1) với stack liên kết hoặc mảng đủ dung lượng. Bộ nhớ tăng theo số nước đi được lưu. Stack không tối ưu cho tìm tùy ý theo tọa độ, nhưng nghiệp vụ hoàn tác không cần thao tác đó. Nếu cần làm lại, có thể bổ sung một stack redo và xóa nó khi có nước đi mới.

Máy tính có ngoặc và độ ưu tiên

Thao tác chính là xử lý toán tử đúng thứ tự ưu tiên và phạm vi ngoặc. Dùng stack toán tử khi chuyển trung tố thành hậu tố. Toán hạng đi vào đầu ra, còn toán tử được đẩy hoặc lấy ra theo độ ưu tiên. Dấu ngoặc mở đánh dấu ranh giới xử lý và dấu đóng kết thúc phạm vi ấy. Với t token, mỗi token được đẩy và lấy hữu hạn lần nên quá trình có thể đạt O(t). Nếu cần in lại cấu trúc hay đánh giá nhiều lần, dựng thêm cây biểu thức như N4. Cây dùng O(t) bộ nhớ và thêm chi phí cấp phát nút. Queue không tự cung cấp quy tắc LIFO cần để xử lý ngoặc lồng nhau.

Danh bạ lớn có mã tăng theo năm

Thao tác quan trọng là tìm hồ sơ theo mã và thêm hồ sơ mới. Dãy mã tăng làm BST thường dễ suy biến. Chọn AVL để giữ chiều cao O(log n). Tra cứu và thêm đều có chi phí xấu nhất O(log n). Khóa phải gắn với dữ liệu hồ sơ và được xử lý nhất quán khi xóa. Đổi lại, mỗi nút lưu thêm height và thao tác cập nhật có thể cần xoay. Mảng đã sắp xếp tìm nhanh nhưng chèn giữa có thể phải dời O(n) phần tử. Khi khối lượng thêm và tìm đều đáng kể, AVL là lựa chọn hợp lý trong các cấu trúc của môn.

Điểm danh 40 người rồi in toàn bộ

Thao tác chính là duyệt danh sách theo thứ tự đã định. Dùng mảng hoặc vector gồm 40 bản ghi để lưu tên và trạng thái có mặt. In toàn bộ mất O(n), cũng là chi phí bắt buộc vì phải xuất đủ n người. Tìm tuần tự một người mất O(n), nhưng n = 40 nên chi phí thực tế nhỏ. Mảng có cách cài đặt đơn giản và giữ thứ tự danh sách lớp. AVL không giảm được độ phức tạp của thao tác in hết. Nó còn cần cấp phát nút, chiều cao và xử lý xoay. Vì vậy không chọn AVL cho nghiệp vụ nhỏ và chủ yếu xuất toàn bộ này.

14 N8 Chương trình từ điển

Mã nguồn: N8_TuDien.cpp. WordNode lưu key và meaning là chuỗi, hai con trỏ và height. Bản BST và bản AVL dùng cùng dữ liệu và cùng so sánh chuỗi; cờ avl quyết định có tái cân bằng hay không. Khi gặp khóa đã có, cập nhật nghĩa, không tạo nút thứ hai.

Dữ liệu và thứ tự chèn

12 từ ban đầu: meo, cho, sach, ban, hoa, ong, xe, ao, ga, lua, nha, tre. Mỗi từ có nghĩa tiếng Việt không dấu trong mã nguồn, ví dụ meo là “dong vat bat chuot”, sach là “tap trang de doc”.

8 từ thêm theo thứ tự alphabet: bo, ca, dao, ech, gio, hat, kem, lon. Đây là các khóa mới và được chèn vào cây đang có 12 từ. Không mặc định rằng thêm một đoạn tăng vào cây bất kỳ sẽ khiến toàn cây thành một nhánh.

Thí nghiệm	Chiều cao BST	Chiều cao AVL
12 từ ban đầu	4	4
Sau khi thêm 8 từ tăng	6	5
Dựng lại từ 20 từ đã sắp xếp	20	5

```
LNR cua tu dien 20 tu
ao ban bo ca cho dao ech ga gio hat
hoa kem lon lua meo nha ong sach tre xe

Tim "lon": gia suc lay thit
So nut tham BST: 6
So nut tham AVL: 4
```

Nhận xét

Cây ban đầu được chọn có thứ tự chèn tương đối thuận lợi nên BST và AVL đều cao 4. Sau tám lần thêm, BST cao 6 còn AVL cao 5. Thí nghiệm đối chứng dựng cây rỗng từ toàn bộ từ đã sắp xếp làm rõ tác động của thứ tự chèn: BST cao 20 trong khi AVL chỉ cao 5.

Với khóa chuỗi, O(h) lần so sánh chưa tính chi phí so sánh ký tự. Nếu độ dài khóa tối đa là L, một thao tác tra cứu có thể tốn O(Lh); AVL cho O(L log n) ở trường hợp xấu. Ví dụ nhỏ này minh họa cấu trúc và số nút thăm, không được coi là số liệu thời gian của hệ thống từ điển 100.000 từ.

Cách dùng cùng báo cáo

Chạy N8_TuDien.cpp để xem đủ nghĩa của 20 từ, số liệu chiều cao ở ba giai đoạn và truy vấn “lon”. Thay đổi hai vector words, more trong main để thử dữ liệu khác. Nếu muốn quan sát xoay với khóa số, dùng chế độ trace trong Bai10_AVL.cpp hoặc chương trình N5_BonXoay.cpp.

15 Câu suy luận C1 đến C11

C1 Dãy LNR. BST không trùng cho dãy tăng nghiêm ngặt. Nếu sai, cần kiểm tra liên kết và thao tác chèn, xóa, xoay hoặc chính hàm duyệt; chỉ nhìn dãy sai chưa kết luận duy nhất lỗi ở hàm nào.

C2 Nút thế mạng. Bài làm chọn min của cây phải. Max của cây trái cũng đúng nếu cài nhất quán. Hai cách có thể cho hình cây khác nhau nhưng giữ cùng tập khóa sau xóa.

C3 Mức và chiều cao. Mức đo từ gốc đến nút, gốc mức 1. Chiều cao đo đường dài nhất từ nút xuống lá theo số nút. Cây một nút có chiều cao 1; cây rỗng 0.

C4 Dấu BF. BF = h(phải) − h(trái). Nút lệch trái có BF âm; AVL hợp lệ khi mọi nút có BF thuộc −1, 0, 1.

C5 Xoay LR. Với 30, 10, 20, xoay phải ngay tại 30 tạo gốc 10 và nhánh phải 30 có con trái 20, vẫn lệch. Phải xoay trái ở 10 rồi phải ở 30. Trong hình tổng quát mục 10, T2 về phải của 10 và T3 về trái của 30.

C6 Các hình cây. Perfect có mọi tầng đầy; complete chỉ cần tầng cuối lấp trái sang phải; skew là một nhánh. Perfect cũng là complete. Dùng 2 nút để minh họa complete nhưng không perfect vì cây complete 3 nút đồng thời perfect.

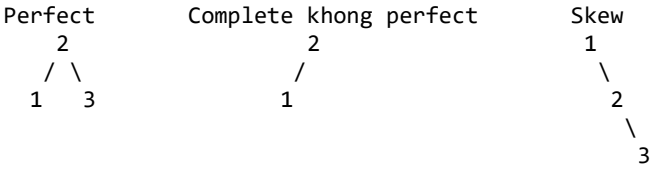
C7 Duyệt theo mức. Dùng Queue. Nếu với mỗi mức lại gọi đệ quy từ gốc để tìm nút ở mức đó, cây skew n nút tốn $1 + 2 + \dots + n = O(n^2)$, trong khi BFS là O(n).

C8 Phản chứng. Chèn 1, 2, …, n tạo BST chỉ có nhánh phải. Tìm n thăm n nút, O(n); do đó "BST luôn O(log n)" là sai.

C9 Đếm lá. Cây gốc 2, trái 1, phải 3, và 3 có phải 4: $n_0 = 2$, $n_1 = 1$, $n_2 = 1$. Có $2 = 1 + 1$. Nút 3 có một con nên không phải lá.

C10 Thứ tự push. Nếu push trái rồi phải, stack lấy phải trước, nên cho thứ tự gốc → phải → trái. Cần push phải trước, trái sau để được NLR.

C11 Khóa trùng. Thêm count giữ mỗi khóa một vị trí và gom tần suất. Tạo hai nút cùng khóa vi phạm quy tắc so sánh nghiêm ngặt của bài, khiến cách tìm/xóa một hay mọi lần xuất hiện không còn rõ. BST cho phép trùng cần quy ước khác, không phải mô hình đang dùng.



16 Câu suy luận C12 đến C22

C12 Tìm trong khoảng. Nếu data < min, bỏ toàn bộ nhánh trái; nếu data > max, bỏ toàn bộ nhánh phải. Nếu trong khoảng, xét hai phía có thể chứa kết quả và in theo LNR. Chi phí O(h + k), xấu nhất O(n).

C13 Phần tử thứ k. Duyệt LNR và dừng ở lần thăm thứ k. Chi phí O(h + k), xấu nhất O(n), bộ nhớ stack O(h). Không có trường kích thước cây con thì không bảo đảm O(log n).

C14 Sao chép. NLR tự nhiên vì tạo gốc rồi gắn bản sao hai cây con. LRN vẫn sao chép được nếu dựng hai cây con trước, sau đó tạo gốc và nối chúng; không có bắt buộc chỉ dùng NLR.

C15 Thời gian AVL. Với n nhỏ và dữ liệu thuận lợi, cập nhật height, nhánh kiểm tra và xoay có thể làm AVL tốn thời gian hơn BST. Cần xem nhiều truy vấn, chiều cao, số nút thăm và phép đo lặp; một lần clock không đủ kết luận.

C16 Phạm vi môn. Chuẩn đầu ra của phần này dừng ở BST và AVL; không thay bài bằng Red-Black Tree.

C17 Biểu thức tiền tự. NLR tương ứng biểu thức tiền tố, chẳng hạn * + 2 3 4. LRN tương ứng hậu tố và phù hợp với tính giá trị từ các con lên gốc.

C18 Height bị lỗi. Dãy 30, 20, 10, 5 là phản ví dụ cụ thể: bỏ cập nhật sau xoay khiến tại gốc 20, BF lưu thành +1 thay vì −1 và h thành 4 thay vì 3. Không nhất thiết đã chọn sai RR ngay ở khóa thứ tư; số liệu sai đủ để bác bỏ cài đặt.

C19 Khóa gần nhất. Với BST đúng và khoảng cách |data − x|, một đường Search cùng ứng viên tốt nhất là đủ. Khi x < data, mọi khóa phía phải còn lớn hơn data nên không thể gần hơn chính data; phía trái tương tự. Không có trường hợp buộc duyệt cả hai cây con cho bài tìm một khóa gần nhất này. Nếu bỏ giả thiết BST thì có: gốc 20, hai con 8 và 10, x = 9 cần xét cả hai để tìm đủ khóa gần nhất.

C20 Chọn cấu trúc. 40 sinh viên cần in danh sách: mảng/vector, O(n) xuất. Hoàn tác mới nhất: Stack, O(1). Quầy vé FIFO: Queue, O(1) thêm/lấy. Từ điển lớn thêm theo alphabet: AVL, O(log n) lần so sánh khóa.

C21 Dựng từ duyệt. NLR cho gốc ở đầu; vị trí gốc trong LNR chia hai cây con, nên hai dãy xác định duy nhất khi khóa phân biệt. Một dãy LNR có thể thuộc nhiều cây khác gốc: 1 2 3 thuộc cả gốc 2 và nhánh phải gốc 1.

C22 Xóa gốc một con. Với 2 → phải 3 → phải 4, xóa 2 trả về con duy nhất 3. Cây mới gốc 3 có con phải 4. Lệnh gọi phải là root = Delete(root, 2) để tránh giữ con trỏ gốc đã giải phóng.

17 Kiểm thử và hướng dẫn sử dụng

Kết quả kiểm thử

Nhóm	Phạm vi kiểm tra	Kết quả
Biên dịch	13 file độc lập, C++11, -Wall -Wextra -pedantic	Đạt
BST	5.000 thao tác chèn/xóa, đối chiếu vector sau mỗi bước	Đạt
AVL	10.000 thao tác chèn/xóa, thứ tự và chiều cao thật mỗi bước	Đạt
Ca biên	Cây rỗng, int nhỏ nhất/lớn nhất, khóa trùng, khóa không có	Đạt
Xoay và xóa	LL, RR, LR, RL; xóa với BF(con)=0; height sai có chủ ý	Đạt
Mở rộng	Hòa khoảng cách, k sai, cây không complete, cân bằng lại	Đạt
File cây	Giữ hình, file rỗng, token sai, tràn int, NLR sai, khóa trùng	Đạt
Tần suất	Xóa hai con khi nút thế mạng có count = 2	Đạt
Dựng cây	Cặp duyệt đúng, rỗng, khác độ dài, mâu thuẫn, trùng	Đạt
Biểu thức	9 ca hợp lệ và sai, gồm trừ/chia để kiểm thứ tự	Đạt
Menu	Nhập chữ, khóa sai, chèn trùng, xóa khóa không có, thoát	Đạt

Bất biến được đối chiếu bằng assert trong bộ kiểm thử nội bộ; giá trị chuẩn dùng vector sắp xếp, không dùng một cây khác làm chuẩn. Kiểm thử chức năng không thay thế công cụ phát hiện lỗi bộ nhớ động; chưa thực hiện đo bằng sanitizer hoặc Valgrind.

Biên dịch và chạy

Mở terminal tại thư mục source. Mỗi file có main riêng, vì vậy biên dịch từng file; không gộp tất cả .cpp vào cùng một chương trình. Cần trình biên dịch hỗ trợ C++11 trở lên. Các lệnh dưới dùng PowerShell trên Windows.

```
g++ -std=c++11 -O2 -Wall -Wextra -pedantic Bai09_BST.cpp -o BST.exe
.\BST.exe
.\BST.exe --demo
g++ -std=c++11 -O2 Bai10_AVL.cpp -o AVL.exe
.\AVL.exe --demo
g++ -std=c++11 -O2 N5_BonXoay.cpp -o N5.exe
.\N5.exe
```

Các chương trình còn lại chạy ví dụ mặc định ngay khi mở. Bai09_MoRong nhận đường dẫn file lưu; file đích được ghi đè khi chạy. N4_CayBieuThuc nhận chuỗi hậu tố đặt trong dấu ngoặc kép. Console dùng tiếng Việt không dấu để tránh lỗi mã hóa trên terminal cũ.

18 Tổng kết và nguồn tham khảo

Kết quả đạt được

Bộ bài làm hoàn thành Bài 9, Bài 10, Bài 11; chín bài mở rộng của BST; phần mở rộng AVL; tám bài N1–N8 và 22 câu C1–C22. Các bài nâng cao kế thừa cách tổ chức nút và con trỏ của giáo trình, đồng thời sửa quy ước và điều kiện thuật toán chưa nhất quán.

BST phù hợp để hiểu tổ chức khóa và thao tác con trỏ, nhưng hiệu quả phụ thuộc chiều cao. AVL bổ sung cơ chế xoay để giữ chiều cao O(log n), đổi lại phải duy trì thông tin height và cập nhật đúng sau mỗi thay đổi. LNR tăng, chiều cao thật và BF tại mọi nút là ba kiểm tra bổ sung cho nhau, không thể dùng một kiểm tra để thay tất cả.

Báo cáo trình bày phần cốt lõi và kết quả; mã nguồn đầy đủ nằm trong 13 file .cpp đi kèm. N1–N8 đều có chương trình minh họa, kể cả N7 vốn cho phép chỉ giải bằng hình. Phần lựa chọn cấu trúc N8 và C1–C22 được giải bằng lập luận trong báo cáo.

Nguồn sử dụng

GiaoTrinhThucHanh_Phan3.pdf, bản có tiêu đề “Giáo trình thực hành cấu trúc dữ liệu và giải thuật, Phần 3 Cấu trúc cây”, ngày ghi trên tài liệu 04 tháng 09 năm 2026, 47 trang. Số trang dẫn trong báo cáo là số trang PDF, trùng số trang in trên tài liệu.

Phạm vi	Trang PDF
Bài 9 và chín bài mở rộng	2–20
Bài 10 và phần mở rộng AVL	20–37
Bài 11	37–40
Bài nâng cao N1–N8	40–46
Câu suy luận C1–C22	46–47

Nhật ký hỗ trợ AI

1. AI đọc giáo trình và lập danh sách yêu cầu Bài 9–11, bài mở rộng, N1–N8 và C1–C22.

2. AI đối chiếu mã mẫu, thống nhất quy ước chiều cao, dấu BF và chỉ ra các ví dụ chưa đúng.

3. AI tạo 13 file C++ và các lời giải, sơ đồ cây, giải thích thuật toán trong báo cáo.

4. AI biên dịch, chạy ví dụ và tạo kiểm thử tự động cho các bất biến, ca biên và đầu vào sai.

5. AI tổng hợp số liệu chạy thực tế vào báo cáo; các hình và lời giải trong tài liệu này không phải bản ghi thao tác vẽ tay của sinh viên.